

Programação Orientada a Objetos com C++

Leonardo Medeiros

Maio de 2024

Objetivos de Aprendizado

- Depois dessa unidade você deve ser capaz de:
 - Usar pilha, fila e fila de prioridade em C++

Tópicos

- Pilha em C++
- Fila em C++
- Fila de prioridade em C++

Adaptador de contêiner em C++

- Os adaptadores de contêineres são uma parte da **biblioteca padrão** do C++ (STL)
- Nos dá uma maneira de modificar ou adaptar as classes de contêineres existentes para atender a necessidades ou requisitos específicos.

O que é um adaptador de contêiner em C++?

- Em C++, os adaptadores de contêiner são interfaces especializadas criadas sobre outros contêineres sequenciais, como deque, vetor ou lista.
- Limitam a funcionalidade em um contêiner preexistente e fornece funcionalidades mais específicas.
- Isso é feito para **evitar** a definição de uma interface completamente nova para contêineres que podem ser criados sobre os contêineres já existentes.

Adaptador de contêiner em C++

- A STL (Standard Template Library, biblioteca de modelos padrão) tem três adaptadores de contêineres principais:
 - Stack (Pilha)
 - Queue (Fila)
 - Priority Queue (Fila de prioridade)

Pilha em C++

- O adaptador de pilha segue o princípio LIFO (Last In, First Out), e a inserção e a remoção de elementos só podem ser feitas a partir do topo do contêiner da pilha.
- A seguir estão as principais operações da pilha:
 - `push(elm)`: Insere o elemento `elm` no topo.
 - `pop()`: Remove o elemento do topo.
 - `top()`: Retorna uma referência ao elemento do topo.
 - `empty()`: Verifica se a pilha está vazia.
 - `size()`: Retorna o número de elementos.

Exemplo de pilha: ex12-01.cpp

- O exemplo abaixo demonstra o uso de operações-chave no adaptador de pilha em C++.

```
// C++ program to demonstrate the usage of key operations in
// stack adapter in C++.
#include <iostream>
#include <stack>
using namespace std;

int main() {
    // Declaring an instance of the stack adapter with
    stack<int> myStack;

    // Push elements onto the stack
    myStack.push(10);
    myStack.push(20);
    myStack.push(30);

    // print size of stack
    cout << "Size of stack is: " << myStack.size() << endl;
```


Exemplo de pilha: ex12-01.cpp (II)

```
// Pop elements from the stack until it's empty
cout << "Elements in a stack are: ";
while (!myStack.empty()) {
    // Display the top element
    cout << myStack.top() << " ";

    // Remove the top element
    myStack.pop();
}
return 0;
}
```

Fila em C++

- O adaptador de fila segue o princípio FIFO (First In First Out, primeiro a entrar, primeiro a sair), o que significa que o elemento é inserido na parte de trás e removido da frente da fila.
- A seguir estão as principais operações da fila:
 - `push(elm)`: Insere o elemento `elm` na parte de trás.
 - `pop()`: Remove o elemento da frente.
 - `front()`: Acessa o elemento frontal.
 - `back()`: Acessa o último elemento.
 - `empty()`: Verifica se a fila está vazia.
 - `size()`: Retorna o número de elementos em uma fila.

Exemplo de fila: ex12-02.cpp

- O exemplo abaixo demonstra o uso de operações-chave no adaptador de fila em C++.

```
// C++ program to demonstrate the usage of key operations in
// queue adapter in C++.
#include <iostream>
#include <queue>
using namespace std;

int main() {
    // Create a queue of integers
    queue<int> myQueue;

    // Add elements to the queue
    myQueue.push(10);
    myQueue.push(20);
    myQueue.push(30);

    // Print the size of the queue
    cout << "Size of queue is: " << myQueue.size() << endl;
```

Exemplo de fila: ex12-02.cpp (II)

```
// Print the elements in the queue
cout << "Elements in a queue are: ";
while (!myQueue.empty()) {
    // Print the front element of the queue
    cout << myQueue.front() << " ";

    // Remove the front element from the queue
    myQueue.pop();
}
return 0;
}
```

Fila de prioridade em C++

- O adaptador de fila de prioridade não segue nenhum dos princípios FIFO ou LIFO.
- Nesse adaptador, a ordenação dos elementos é feita com base na prioridade (usando max-heap), de modo que o elemento mais alto está sempre presente na frente.
- A seguir estão as principais operações da fila de prioridade:
 - `push(elm)`: Insere o elemento `elm`.
 - `pop()`: Remove o elemento superior.
 - `top()`: Retorna uma referência ao elemento do topo.
 - `empty()`: Verifica se a fila de prioridade está vazia.
 - `size()`: Retorna o número de elementos.

Exemplo de fila de prioridade:ex12-03.cpp

- O exemplo abaixo demonstra o uso de operações-chave no adaptador de fila de prioridade em C++.

```
// C++ program to demonstrate the usage of key operations in
// priority queue adapter in C++.
#include <iostream>
#include <queue>
using namespace std;
int main() {
    // Create a priority queue of integers (default is a
    // max-heap)
    priority_queue<int> myPriorityQueue;

    // Add elements to the priority queue
    myPriorityQueue.push(30);
    myPriorityQueue.push(10);
    myPriorityQueue.push(20);

    // Print the size of the priority queue
    cout << "Size of priority queue is: "
         << myPriorityQueue.size() << endl;
```

Exemplo de fila de prioridade:ex12-03.cpp (II)

```
// Print the elements in the priority queue
cout << "Elements in a priority queue are: ";
while (!myPriorityQueue.empty()) {
    // Print the top (maximum) element of the priority
    // queue
    cout << myPriorityQueue.top() << " ";

    // Remove the top element from the priority queue
    myPriorityQueue.pop();
}
return 0;
}
```

Sumário, Atividades e Dúvidas

- Pilha, Fila e Fila de Prioridade
- Perguntas?
- Atividade 1: Escreva um programa C++ que use `std::stack`
- Atividade 2: Escreva um programa C++ que use `std::queue`
- Atividade 3: Escreva um programa C++ que use `std::priority_queue`
- Anotem e tragam as dúvidas na próxima aula

Referências

- GeeksForGeeks. <https://www.geeksforgeeks.org/>

Obrigado!